Cooperative Path Planning With Asynchronous Multiagent Reinforcement Learning

Jiaming Yin, Weixiong Rao, *Member, IEEE*, Yu Xiao, *Member, IEEE*, and Keshuang Tang

***Abstract*—As the number of vehicles grows in urban cities, planning vehicle routes to avoid congestion and decrease commuting time is important. In this paper, we study the shortest path problem (SPP) with *m*ultiple *s*ource-*d*estination pairs, namely MSD-SPP, to minimize the average travel time of all routing paths. The asynchronous setting in MSD-SPP, i.e., vehicles may not simultaneously complete routing actions, makes it challenging for cooperative route planning among multiple agents and leads to ineffective route planning. To tackle this issue, in this paper, we propose a two-stage framework of inter-region and intra-region route planning by dividing an entire road network into multiple sub-graph regions. Next, the proposed asyn-MARL model allows efficient asynchronous multi-agent learning by three key techniques. First, the model adopts a low-dimensional global state to implicitly represent the high-dimensional joint observations and actions of multi-agents. Second, by a novel trajectory collection mechanism, the model can decrease the redundancy in training trajectories. Additionally, with a novel actor network, the model facilitates the cooperation among vehicles towards the same or close destinations, and a reachability graph can prevent infinite loops in routing paths. On both synthetic and real road networks, the evaluation result demonstrates that asyn-MARL outperforms state-of-the-art planning approaches.**

***Index Terms*—Shortest path, route planning, multi-agent reinforcement learning.**

I. Introduction

RECENT years have witnessed the ever-growing number of vehicles in urban cities. This rising number contributes to traffic congestion, high commuting time [15], and air pollution (as exhaust emissions [17]). For example, it is reported that each driver in New York City lost 101 hours in congestion during 2023 [16]. Thus, it is important to plan vehicle routes to avoid traffic congestion and decrease driving time. The classic shortest path problem (SPP) aims to determine the route with the minimal travel time or path distance of a given source-destination pair. In real-world applications, it is common to plan the shortest paths for multiple source-destination pairs (MSD), namely MSD-SPP.

Received 2 May 2024; revised 29 November 2024; accepted 2 January 2025. Date of publication 8 January 2025; date of current version 7 May 2025. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB4301904, Grant 2022YFE0208000, in part by the National Natural Science Foundation of China under Grant 52372319, and in part by the Shanghai Key Lab of Vehicle Aerodynamics and Vehicle Thermal Management Systems. Recommended for acceptance by H. Shen. *(Corresponding author: Weixiong Rao.)*

Jiaming Yin, Weixiong Rao, and Keshuang Tang are with Tongji University, Shanghai 200070, China (e-mail: wxrao@tongji.edu.cn).

Yu Xiao is with Aalto University, 02150 Espoo, Finland (e-mail: yu.xiao@aalto.fi).

Digital Object Identifier 10.1109/TMC.2025.3526979

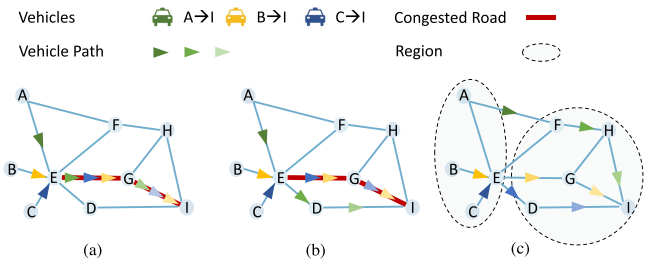


Fig. 1. Illustration of three route planning approaches. (a) Entire path-based, (b) next-hop node-based, and (c) our approaches. Each node represents an intersection and each edge represents a road segment.

Unlike the classic SPP, the challenge of solving MSD-SPP is that we have to consider the traffic capacity constraints of road networks. When the number of vehicles on a certain road segment exceeds the road capacity constraint, traffic congestion occurs, leading to increased travel time. Given the capacity constraints of road networks, MSD-SPP aims to plan the shortest paths for all source-destination pairs with the objective of minimizing an aggregate metric, such as the average travel time.

Though the classic SPP problem has been well studied in the literature, solving MSD-SPP is hard. Even a simplified variant of the problem, e.g., the k-Disjoint Shortest Path problem (kDSP), by considering only two source-destination pairs, has been proved to be NP-complete [8]. Here, the kDSP aims to find disjoint shortest paths for k source-destination pairs to alleviate congestion on graph edges. In real-world scenarios with a great number k, solving kDSP becomes much harder. It is particularly true since road segments need to accommodate multiple vehicles simultaneously while subjecting to capacity constraints.

To illustrate the challenges inherent in the MSD-SPP, we present two baseline approaches. (1) The *Entire Path-based approach* independently plans the entire routing path for each individual vehicle. For example, we can exploit a classic algorithm such as Dijkstra [7] and A* algorithms [12] to compute the shortest path for a given source-destination pair. However, when considering multiple source-destination pairs in the context of MSD-SPP, this approach may result in multiple shortest paths that share certain road segments. When the number of vehicles on these road segments exceeds the road capacity constraints, traffic congestion occurs. In Fig. 1(a), all three vehicles are routed to the road segments $E \rightarrow G \rightarrow I$ simultaneously, and these road segments may become congested.

(2) The *Next-hop node-based approach* re-plans routes for individual vehicles upon reaching intersections to select the

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

next-hop ones based on the real-time traffic conditions. The studies [11], [43] exploit Deep reinforcement learning (DRL) algorithms to learn such routing policies. The DRL-based routing policies enable adaptive decision-making in response to dynamically changing traffic conditions. As illustrated in Fig. 1(b), when the green car arrives at the intersection node E, a DRL algorithm may observe congestion on the road segment $E \rightarrow G$ and then choose the intersection D as the next intersection. However, it still does not effectively handle the situation where multiple vehicles simultaneously enter an intersection. Again, in Fig. 1(b), when the yellow and blue cars arrive at the node E simultaneously, the DRL model might select intersection G as the next hop for both cars, thereby potentially exacerbating congestion on the road segment $E \rightarrow G$. Another limitation of this approach is that it may generate infinite loops due to lacking a global view.

Some recent studies exploit multi-agent reinforcement learning (MARL) to enable cooperation among multiple agents by treating either vehicles or intersections as agents [2], [41]. Such cooperation offers promising results in planning vehicle routes. However, these works still suffer from two following issues.

- The MSD-SPP is an *asynchronous* decision-making problem where vehicles may not arrive at intersections simultaneously. Assuming each vehicle is treated as an agent, in Fig. 1(b), when the yellow and blue cars have already reached the intersection E, the agents of the two vehicles need to decide their next-hop nodes. Yet, the green car is still on its way to E, and no decision is required until its arrival at E. However, existing MARL methods do not work well in the asynchronous MSD-SPP setting: they often assume that all agents make *synchronous* actions and collect joint observations and actions at each time step. When some vehicles are still on the way to the next intersection, the action and observation information synchronously gathered at each time step may remain unchanged. This leads to high redundancy within training trajectory data, making the MARL training inefficient [44].
- When the number of intersections or vehicles is high, existing works using intersections or vehicles as multi-agents encounter the *scalability* issue and the MARL algorithms struggle to perform effectively on real-world road networks with thousands of intersections and vehicles.

Until now, the literature works above suffer from the ineffectiveness caused by the asynchronous decision-making setting and the scalability issue caused by a high number of intersections or vehicles. To tackle these issues, this paper proposes an effective and efficient framework to solve MSD-SPP. First, in terms of scalability issue, we propose a two-stage route planning framework. Specifically, we divide the graph representing the original road network into sub-graphs, also known as regions. By modeling each region as an agent, we exploit MARL to develop a region-level route planning approach. That is, for every SPP task within MSD-SPP starting from a given region, the route planning consists of two stages. 1) In the *inter-region route planning*, the agent of the starting region chooses a cutting edge from the current region to a neighbor one, and 2) the *intra-region route planning* plans the sub-path from the source node towards the entry point of the selected cutting edge. We repeat the route planning steps above until the arrival at the final destination. In this way, we decompose the original MSD-SPP into the problem on small regions and can scale to large graphs with hundreds or thousands of nodes.

Given the two-stage route planning framework, we may exploit an existing MARL algorithm to comfortably perform the intra-region route planning and choose desirable sub-paths within divided regions. The key to solve the MSD-SPP then becomes how to choose cutting edges across regions to connect sub-paths, i.e., the inter-region route planning. To this end, we develop a novel asynchronous MARL framework, namely asyn-MARL, to make cooperative inter-region route planning. The asyn-MARL model builds three following components. 1) To address the asynchronous issue and maintain the stability of MARL training, we extend the classic Centralized Training Decentralized Execution (CTDE) approach [25] to the asynchronous settings. We design a low-dimensional global state to implicitly represent the high-dimensional features of joint observations and actions of multi-agents during the MARL training phase. Additionally, to decrease the redundancy in the MARL training trajectories, we develop an asynchronous MARL training trajectory collection mechanism. 2) We employ a novel actor network to enable the cooperation of vehicles and avoid the plan of too many vehicles on the same road segments. In the actor network, a Gated Recurrent Unit (GRU) [5] module is applied to extract the features about vehicle competitions among those within the routing requests at the same time received by a certain region agent. 3) To mitigate infinite loops during the cooperative route planning, we design a reachability graph to prune actions that may lead to loops. In summary, we make the following contributions.

- We propose a scalable route planning framework with two stages of inter-region and intra-region planning schemes. By extending the fine-grained next-hop node-based planning model, the proposed region-level route planning framework performs efficiently on large road networks.
- We develop an asyn-MARL model to make cooperative route planning. Asyn-MARL consists of a carefully designed global state to represent the joint observations and actions, an asynchronous trajectory collection mechanism to decrease redundant trajectories, a novel actor network and a reachability graph to mitigate traffic congestion. To the best of our knowledge, this is the first asynchronous MARL route planning model to solve MSD-SPP.
- We conduct experiments on both synthetic and real-world traffic networks with a microscopic traffic simulation environment SUMO [22]. Evaluation results demonstrate that our approach outperforms traditional SPP algorithms and state-of-the-art MARL approaches regarding throughput, travel time, and CO2 emission volume.

The rest of this paper is structured as follows. Section II first reviews background and related works. Section III then presents the problem definition and the overall framework. Section IV next delves into details of our proposed solution. After that, Section V evaluates our work, and Section VI finally concludes the paper.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

II. Background and Related Works

A. Background

In this section, we briefly review some background of MARL and approaches to train the agents in a multi-agent environment.

Multi-agent System. A multi-agent system consists of an environment and multiple decision-making agents. This process can be formalized by a partially observable Markov game $\langle \mathcal{S}, \mathcal{O}, \mathcal{A}, R, T, \gamma \rangle$ [24]. Here, $\mathcal{S}$ is the set of states. $\mathcal{A}^1, \ldots \mathcal{A}^M$ and $\mathcal{O}^1, \ldots, \mathcal{O}^M$ are action and observation sets of M agents, respectively. Each agent i can receive an observation $o^i \in \mathcal{O}^i$, which contains partial information from the global state $s \in \mathcal{S}$. The transition function T determines the transition probability from the current state $s \in \mathcal{S}$ to the next one $s' \in \mathcal{S}$ given the joint actions of all agents. The reward function R^i for each agent determines the immediate reward that an agent would receive when executing an action a^i. The action a^i typically depends on the global state and the actions of all agents. The the discount factor $\gamma \in [0, 1]$ balances the trade-off between immediate and future rewards. Each agent learns a policy π^i, which maps the agent observation to a distribution over its action set. The optimization objective of each agent is to maximize its expected discounted return $\mathbb{E}_{a,s} \sum_{t=0}^{\infty} \gamma^t r_t^i(s_t, a_t^i, \ldots, a_t^M)$.

Policy Gradient Methods: Reinforcement learning (RL) learns the policy that maximizes the expected discounted returns. Policy gradient estimates the gradient of an agent's expected return with respect to policy parameters, i.e., $\nabla_\theta J(\pi_\theta) = \nabla_\theta \log(\pi_\theta(a_t|s_t)) \sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'}(s_{t'}, a_{t'})$. However, the term $\gamma^{t'-t} r_{t'}(s_{t'}, a_{t'})$ may lead to a high variance.

The *actor-critic* network tackles the high variance issue above by an estimator (i.e., the critic) to approximate the expected returns. One specific actor-critic network [29] approximates the value of a state s_t with a critic model. By computing the advantage $A(s_t, a_t) = r(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t)$, we can update the actor with the policy gradient estimated with the advantage, and then update the critic model by minimizing the temporal-difference error. In addition, some policy gradient methods, e.g., Proximal Policy Optimization (PPO) [28], ensures a stable update of the policy by bounding the policy parameter updates to a trust region. PPO achieves this goal with a novel optimization function, by limiting how much the policy can change in one update.

Cooperative MARL: To learn the optimal policy in a multi-agent system, a straightforward approach is to train each agent independently and maximize its individual reward by treating other agents as part of the environment [34]. Traditional RL typically assumes that the environment is stationary, i.e., the dynamics of the environment remain constant over time. Yet, the agent actions in MARL affect the environment, which in turn influences the behavior of other agents. This interaction between agents introduces *non-stationarity* into the environment, as the environment changes caused by the actions of all involving agents. Due to the non-stationarity issue, the Markov assumption that the current state of the environment contains all the information necessary to make decisions about future actions, does not hold in the context of MARL [26].

Centralized learning and decentralized execution (CTDE) [25] is a widely used strategy to tackle the non-stationarity issue. It allows information to be shared during training, while policies are only conditioned on the local observations of individual agents during the execution stage. CTDE is frequently used in the two categories of MARL methods, centralized policy gradient [9], [23], [44] and value function factorization [27], [31], [33].

- Centralized policy gradient involves decentralized actors for each agent and a centralized critic, where a value function is optimized based on the shared information from all agents. By a centralized critic with the joint observations and actions of all agents as input, Deep Deterministic Policy Gradient (DDPG) policy [21] ensures a stationary environment even if the policies of other agents change. With a centralized critic, the on-policy RL algorithm PPO [28] performs well in cooperative multi-agent settings.
- Value function factorization methods such as VDN [33] and QMIX [27] train decentralized policies in a centralized fashion. Such policies frequently employ a network to estimate joint action-values as a combination of per-agent values that condition only on local observations.

B. Asynchronous MARL

The existing MARL methods above assume that all agents synchronously perform actions at each time step. Yet, in typical vehicle routing tasks, this assumption does not hold. Instead, agents frequently make *asynchronous* decisions. Thus, existing methods do not work well in the asynchronous setting due to the redundancy of joint observations and actions within the training trajectory, i.e., a single observation may correspond to multiple highly diverse actions, or conversely, several observations may correspond to the same action.

To address this issue, some asynchronous MARL methods are developed. The work [40] develops an asynchronous MARL model to solve the bus fleet control problem. It designs a critic to effectively estimate the contribution of the other agents in the asynchronous setting. The recent work [45] considers the cooperative exploration of robots. To address the issue that robots accomplish actions at different time steps, this work extends multi-agent PPO to the asynchronous setting and applies action-delay randomization to improve the generalizability of learned policies to action delays in the real world. In the cooperative e-vehicle charging problem, the work [20] proposes to collect the transition trajectory of an agent based on its own local clock and uses the concatenation of the locations and remaining charging time of other agents as the state.

Nevertheless, these previous works above do not perform well in our asynchronous MSD-SPP setting. First, these works are essentially scenario-specific and are inapplicable to our MSP-SPP problem due to the significantly different observation space. Second, unlike these previous works, the asynchronous MSD-SPP problem suffers from two following issues: 1) the number of vehicles arriving at intersections or regions differs depending upon time steps and associated agents, and 2) the

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

number or dimensionality of decision actions also dynamically differs depending upon time steps and associated agents. How to address such issues in our setting is non-trivial.

C. Vehicle Shortest Path Planning

Existing works on shortest path planning for vehicle navigation typically aim to plan shortest paths and meanwhile avoid traffic congestion. Depending on whether or not the route planning cooperates among vehicles, we divide these works into two categories.

Non-cooperative route planning approaches compute the SPP for an individual vehicle without considering concurrent planning for other vehicles. The classic approaches, Dijkstra algorithm [7] and A* algorithm [12], compute the shortest path between a source-destination pair on a road network. Recently, to solve the classic combinatorial optimization problem, e.g., Travel Salesman Problem (TSP) and Vehicle Routing Problem (VRP), the machine learning-based approaches learn an approximation function that maps input road networks to travel tours [19], [39]. Nevertheless, whenever either graph typologies or edge weights of road networks change, these algorithms must re-plan the solution paths on such dynamic road networks, leading to high computing overhead.

DRL-based approaches iteratively re-plan the next-hop intersection, when a vehicle arrives at an intersection. Our previous work [11] employs a dueling deep Q-network to determine the shortest path routing on a grid road network. The work [42] exploits a graph convolution network and a deep Q-network to perform the shortest path routing on dynamic graphs. Unlike the simple SPP problem, our previous work [43] studies the NP-hard problem, the constrained shortest path, on dynamic graphs.

All the works above *independently* compute the shortest paths for an individual vehicle without cooperation with other vehicles. Such paths could lead to traffic congestion on specific road segments if an excessive number of vehicles are unfortunately planned onto the same road segments.

Cooperative route planning approaches consider the cooperation among vehicles. Some methods try to calculate paths of all the vehicles together to achieve the optimal average travel time. Gawron's algorithm [10] is one of the most popular methods for traffic assignment. Given the traffic demand between intersections, it computes the fastest route for each vehicle in each iteration. Then, it assigns a cost to each road segment based on the traffic intensity. By iteratively moving some vehicles to less congested paths and re-computing road costs, a user equilibrium is finally achieved. However, this method is computationally expensive due to the iterative steps. Instead of computing all routing paths directly, some works [13], [36] develop an RL policy to assign edge weights, which can be converted into flow ratios on graph edges with softmin operation. The two works greatly reduce the solution space from $V(V-1)E$ to Em, where V and E are the numbers of vertices and edges, respectively. Nonetheless, they still suffer from scalability because training DRL policies remains challenging for large graph sizes due to the extensive action space.

TABLE I
SUMMARY OF SYMBOLS

Symbol	Definition
$\mathcal{G}=(\mathcal{V},\mathcal{E})$	road network $\mathcal{G}$, intersection set $\mathcal{V}$, road segment set $\mathcal{E}$
v_i, e_{ij}, c_{ij}	intersection v_i, road segment e_{ij} of $v_i \rightarrow v_j$, capacity c_{ij}
l, L	the vehicle l starting from v_l^s to v_l^d at time t_l^s, # of vehicles
p_l, t_l^s, t_l^d	routing path p_l, departure time and arrival time of vehicle l
M, R_i, π_i	# of divided regions, the i-th region (agent) and policy of R_i
$\mathcal{E}_i^c$, $\mathcal{V}_i^c$	cutting edges from R_i and boundary nodes in R_i
$\mathcal{G}_l^c$, $\mathcal{G}_l^r$	connection graph and reachability graph of vehicle l
$q_t^{i,k}$, $\mathcal{Q}_t^i$	the k-th request and request set received by R_i at time t
$o_t^{i,k}$, $a_t^{i,k}$	observ. and action of agent R_i for $q_t^{i,k}$ at time t
$o_t^{i,\mathcal{G}}$	road network observ. of agent R_i at time t
o_t^{i,q_k}	routing request observ. of $q_t^{i,k}$
$s_t^{i,k}$, $r_t^{i,k}$	state and reward of agent R_i for $q_t^{i,k}$ at time t
F, F', D_h	dim. of edge and request feature vectors, dim. of embeddings
$e_t^{i,\mathcal{G}}$	road network embedding of R_i at time t
e_t^{i,q_k}, $e_t^{i,\mathcal{Q}}$	embedding of request $q_t^{i,k}$ and all requests in $\mathcal{Q}_t^i$
$u_t^{i,k}$	scores on action roads of agent R_i for $q_t^{i,k}$ at time t
$\mathcal{D}_i$, $\mathcal{D}$	data buffer of agent R_i and centralized data buffer
E, T	No. of episodes, and No. of simulation steps in a episode
θ_i, ϕ	parameters of agent R_i's policy and the critic
$\hat{G}_t^{i,k}$, $\hat{A}_t^{i,k}$	discounted return and advantage of action $a_t^{i,k}$

Instead of computing all shortest paths together, MARL-based methods attempt to learn decentralized routing policies with multiple vehicle or intersection-based agents. Since the action space of a single agent is much smaller than the original solution space, it is feasible for MARL to train such policies efficiently.

Depending on how to model agents in the MARL setting, these methods can be split into two categories. (1) Some works assign agents to individual *vehicles*. Treating each vehicle as an agent, the work [30] develops a mean field multi-agent deep Q learning to update its en-route path choice. Another work [41] learns a routing policy by incorporating a transformer network [37] into the actor model to select the next-hop road segment. The previous work [46] proposes a stage learning algorithm to learn the γ-Nash equilibrium in the MSD-SPP setting. Yet, these methods work well only with a few vehicle agents (typically in tens). (2) Some works instead treat each *intersection* as an agent and provide routing decisions for incoming vehicles. For instance, the work [2] assigns an agent to each road intersection and selects a next-hop road segment when a vehicle arrives. The work [32] makes cooperative control decisions for traffic signal lights and autonomous vehicles using a hierarchical RL framework.

These MARL works typically make control decisions by choosing next-hop intersections or road segments. Due to the asynchronicity in MSD-SPP, it is hard to directly exploit the CTDE framework to solve the non-stationarity issue during MARL training. Moreover, existing MARL-based methods cannot scale to large graphs because it is impossible to train and deploy too many vehicle or intersection agents.

III. Problem Definition and Overview

In this section, we first give our problem definition, then highlight the challenges, and present an overview of our solution framework. Table I lists the definitions of the symbols.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

A. Problem Definition

Definition 1. Road Network: A road network, modeled by a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, consists of a node set $\mathcal{V}$ and edge set $\mathcal{E}$. Each node $v_i \in \mathcal{V}$ denotes a road intersection. An edge $e_{ij} \in \mathcal{E}$ indicates the road segment from the node v_i to another one v_j. Denote c_{ij} to be the traffic capacity constraint of e_{ij}, and n_t^{ij} the number of vehicles on the road segment e_{ij} at time t, yet V_{ij}^{max} the maximal speed on e_{ij}. When $n_t^{ij} \leq c_{ij}$, for simplicity, we assume that the vehicles move with the maximal speed V_{ij}^{max}, and otherwise a slower travel speed by a fraction $\frac{\alpha \cdot c_{ij}}{n_t^{ij}}$ with $0 < \alpha < 1$. A greater number n_t^{ij} over the capacity c_{ij} leads to a lower travel speed along the road. We then compute the travel duration d_t^{ij} by the division between the road length h_{ij} and the travel speed.

Definition 2. Vehicle: Given L vehicles in a road network $\mathcal{G}$, each vehicle l is represented with a triplet $\langle v_l^s, v_l^d, t_l^s \rangle$, where v_l^s and v_l^d are source and destination intersection nodes, respectively, and t_l^s indicates the departure time. The routing path of the vehicle consists of a sequence of nodes $p_l = [v_l^s, \ldots, v_l^d]$.

Problem 1. MSD-SPP: We formulate MSD-SPP as an optimization problem. By learning a route planning policy π_θ, we expect to minimize the average travel time of all vehicles.

$$\min_\theta \frac{1}{L} \sum_{l=1}^{L} \left(t_l^d - t_l^s\right) \tag{1a}$$

$$p_l = \pi_\theta(v_l^s, v_l^d, t_l^s, \mathcal{G}) \tag{1b}$$

$$t_l^i = t_l^{i-1} + d_{t_l^{i-1}}^{v_l^{i-1} v_l^i}, \;\; t_l^1 = t_l^s, \;\; t_l^d = t_l^{|p_l|} \tag{1c}$$

$$d_t^{ij} = \begin{cases} \frac{h_{ij}}{V_{ij}^{max}}, & \text{if } n_t^{ij} < c_{ij} \\ \frac{h_{ij}}{V_{ij}^{max} \cdot \frac{\alpha \cdot c_{ij}}{n_t^{ij}}}, & \text{otherwise} \end{cases} \tag{1d}$$

In the problem above, the routing policy π_θ parameterized with θ makes decision for each vehicle l to choose the path p_l (1b). Denote v_l^{i-1} and v_l^i to be $(i-1)$th and ith nodes within the path p_l, and we then compute the arrival time t_l^i of vehicle l at node v_l^i by $t_l^i = t_l^{i-1} + d_{t_l^{i-1}}^{v_l^{i-1} v_l^i}$ (1c). Here, the travel duration $d_{t_l^{i-1}}^{v_l^{i-1} v_l^i}$ is computed by the division between the road length and the travel speed of l (1d).

B. Challenges

MSD-SPP involves a trade-off between minimizing the travel time of individual vehicles and meanwhile preventing traffic congestion from planning too many vehicles onto the same roads. Thus, it is essential to take into account vehicle cooperation without breaking road capacity constraints. We aim to learn cooperative policies by MARL. Nevertheless, solving MSD-SPP with MARL is challenging due to the following issues.

1) Ineffectiveness Issues: The cooperation of MARL to solve the asynchronous MSD-SPP is hard. First, the widely used CTDE framework assumes a synchronous MSD-SPP setting. Second, the multi-agent cooperation of vehicle routing plan is essential to optimize the overall traffic. If not well planned, too many vehicles share the same road segments on their shortest paths towards the destination, suffering from traffic congestion on such road segments. Finally, when we exploit MARL to solve the MSD-SPP problem, the complete routing path of a vehicle is composed of the sequential decision result from multi-agents. Such composition could lead to infinite loops when incorrect decisions are made.

2) Inefficiency Issues: First, the MSD-SPP is NP-complete. The classic approach, e.g., the Gawron algorithm [10], is computationally expensive. Second, solving MSD-SPP with MARL, by modeling vehicles or intersections as multi-agents, may lead to a large number of agents and exponential joint observation and action space. The CTDE framework then becomes inefficient.

C. Overall Framework

As shown in Fig. 2, we divide the input road network into M regions, and solve MSD-SPP with a two-stage framework of the *inter-region* and *intra-region* planing. For the *inter-region* route planning, we treat each region as an agent and train such multi-agents with MARL. Within the MARL model, each region agent selects a cutting edge towards a neighboring region by performing the *inter-region* route planning asynchronously. Once the inter-region routing plan (i.e., a cutting edge) is made, we then plan the *intra-region route* to reach the cutting edge. Since each region agent can fully observe the detailed traffic conditions within the region, we then can exploit our previous work [43] to plan the intra-region route. Since the number of regions is significantly smaller than the number of intersections, we expect that the computational cost of the two-stage framework is much lower than the original one. Note that an agent in the rest of the paper by default refers to a region agent, differing from those vehicle or intersection agents in the previous works [2], [41].

In terms of road network partitioning, we divide the input road network into multiple disjoint sub-graphs (e.g., regions) with similar size, while minimizing the number of cutting edges across regions. For each region R_i, we denote $\mathcal{E}_i^c$ to be the *set of cutting edges* originating from R_i, and $\mathcal{V}_i^c$ to be the *set of boundary nodes* within the region R_i that are connected by the cutting edges $\mathcal{E}_i^c$. In Fig. 2(b), we have $M = 3$ regions. The region R_1 involves three cutting edges $\mathcal{E}_1^c = \{v_1 \to v_4, v_3 \to v_5, v_2 \to v_8\}$ and three boundary nodes $\mathcal{V}_1^c = \{v_1, v_2, v_3\}$. We can exploit existing algorithms, such as the classic work METIS [18], to perform graph partitioning, and tune the number of divided regions depending on the capacity of region agents. By assuming that an agent can observe the entire region, including at most E_r edges, we can roughly compute the number of divided regions by $\lceil \frac{E}{E_r} \rceil$ where E is the total number of edges in the input graph $\mathcal{G}$.

Given the M regions, we then treat each region as an individual agent and assume that each agent can observe the following information: (1) the *static information of an entire road network*, such as the GPS coordinates of intersections and the length of all road segments in the road network, (2) the *detail information within the region*, e.g., the average speed and travel time of every road segment in the region, (3) the *information of cutting*

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

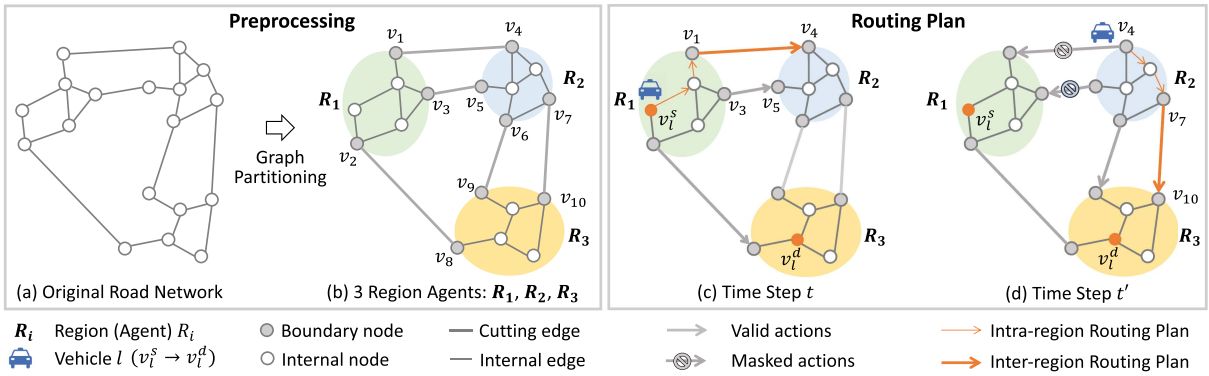


Fig. 2. Pipeline of the proposed method.

edges, such as the average speed and number of vehicles on the cutting edges that originate from the region, and (4) the *estimation information of neighbouring regions*, i.e., the roughly estimated number of vehicles and average vehicle speed of such a neighbour.

Given the observation, each region agent can compute the internal routes within the region and plan the routes across divided regions by the cooperation with other agents. In Fig. 2(c), when the vehicle l starts at v_l^s within the region R_1, the region agent first performs the inter-region plan to enter the neighbouring region R_2 via a selected cutting edge $v_1 \to v_4$, and then computes the intra-region route from v_l^s to v_1. When the vehicle enters the chosen region R_2, the region agent next chooses the cutting edge $v_7 \to v_{10}$ and plans the intra-region route within R_2. After the vehicle enters the region R_3, the associated agent finally plans the intra-region route within R_3 from v_{10} to the destination v_l^d.

Inter-region route planning: We formulate the inter-region route planning as a decentralized partially observable Markov decision process (Dec-POMDP).

- *Agent:* We model each region as an individual agent and denote the set of M agents by $\mathcal{R}$. When a vehicle l starts a trip from a source node or arrives at a new region, it sends a plan request involving the trip to the associated region agent. The request consists of the current node and destination node of the vehicle l.
- *Action:* Suppose that a region agent R_i receives a set of plan requests $\mathcal{Q}_t^i$ sent by vehicles at time step t. For a certain request $q_t^{i,k}$, the agent R_i makes an action $a_t^{i,k}$ to select the next visiting region. Practically, the agent R_i selects one cutting edge from the set $\mathcal{E}_i^c$ as the action $a_t^{i,k}$. Given M agents and $\sum_{i=1}^{M} |\mathcal{Q}_t^i|$ requests, we have joint actions $a_t = (a_t^{1,1}, \ldots, a_t^{M,|\mathcal{Q}_t^M|})$. Here, the size of the joint actions dramatically differs from the time step t.
- *Observation:* The observation $o_t^{i,k}$ of an agent R_i involves the two following parts. First, the *road network observation* $o_t^{i,\mathcal{G}} \in \mathbb{R}^{|\mathcal{E}_i^c| \times F}$ consists of the number $|\mathcal{E}_i^c|$ of F-dimensional feature vectors regarding the cutting edges $\mathcal{E}_i^c$. Each F-dimensional vector involves the features of a cutting edge (i.e., the coordinates of starting and ending nodes, length and travel time of this edge) and traffic estimation (i.e., the estimated number of vehicles and average vehicle speed) of the neighbour region to which the cutting edge is connected. Second, the *plan request observation*$o_t^{i,\mathcal{Q}} \in \mathbb{R}^{|\mathcal{Q}_t^i| \times F'}$ indicates the observation of the requests $\mathcal{Q}_t^i$, where F' is the dimensionality of an individual feature vector. That is, for each request $q_t^{i,k} \in \mathcal{Q}_t^i$, the associated observation $o_t^{i,q_k} \in \mathbb{R}^{F'}$ involves the following items, (1) the coordinates of the current intersection and final destination of the request $q_t^{i,k}$, (2) the least travel time from the source v_c in the request $q_t^{i,k}$ to the cutting edges $\mathcal{E}_i^c$ (computed by the detailed intra-region traffic information at time step t), and (3) the length of the shortest path among those paths from the cutting edges $\mathcal{E}_i^c$ to the destination in the request $q_t^{i,k}$ on the static road network.
- *Policy:* Given the observation $o_t^{i,k}$ above, the policy π^i gives a probability $\pi^i(a_t^{i,k}|o_t^{i,k})$ that the agent R_i makes an action $a_t^{i,k} \in \mathcal{E}_i^c$. We denote the joint policy of M agents as $\boldsymbol{\pi} = (\pi^1, \ldots, \pi^M)$.
- *Reward:* We define the local reward of agent R_i to make the action $a_t^{i,k}$ for the request $q_t^{i,k}$ by $r_t^{i,k} = -(t' - t)$. Here, t' is the time step when the vehicle arrives at the end of the road segment chosen by the action $a_t^{i,k}$ or finally reaches the destination. In Fig. 2(c), when a vehicle l starts from the source v_l^s and sends a plan request, the agent R_1 receives the request and then selects the cutting edge $v_1 \to v_4$ as the action. The reward in this case is the negative value of the travel time of this vehicle l on the subpath $v_l^s \rightsquigarrow v_4$ intermediately through the region R_1. Next, when the region agent R_2 selects the cutting edge $v_7 \to v_{10}$ as an action, where v_{10} is inside the destination region, we similarly compute the reward of the action $v_7 \to v_{10}$ as the negative travel time on the subpath $v_4 \rightsquigarrow v_l^d$ passing through the edge $v_7 \to v_{10}$. Since higher travel time leads to a smaller reward, the reward definition is consistent with the objective of the MSD-SPP problem to minimize the average travel time.

Intra-region route planning: Since the intra-region routing plan does not involve multi-agent cooperation, we exploit our previous work [43] to find the shortest path within a region from the current node toward a chosen cutting edge. By assuming that the traffic details such as traffic speed and travel time of all road

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

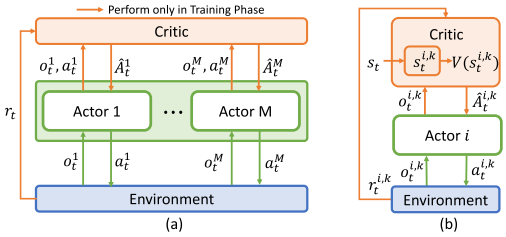


Fig. 3. Multi-agent Actor-Critic framework, where each actor corresponds to a region agent: (a) the standard CTDE scheme, and (b) the asynchronous scheme.

segments inside the region can be fully observed, this method selects the next-hop intersection whenever a vehicle arrives at an intersection, based on the current traffic.

D. Overview of Inter-Region Routing Plan

Given the Dec-POMDP formulation above, we exploit an improved Actor-Critic (AC) network [29] to train the policy of the inter-region route planning model with the CTDE paradigm [44]. Each agent R_i has an *actor* i to make inter-region routing decisions. The general CTDE framework assumes the synchronous decision-making, where each agent makes an action at every time step, as illustrated in Fig. 3(a). We improve the CTDE framework in the asynchronous MSD-SPP setting as shown in Fig. 3(b). At each time step, every *actor* i interacts with the environment in both the training and execution phases, by taking the local observation $o_t^{i,k}$ as input and making an action $a_t^{i,k}$. Instead, the centralized *critic* (i.e., using a neural network as a value estimation function) works only in the training phase. After receiving the reward $r_t^{i,k}$ for the action $a_t^{i,k}$, the critic evaluates the value of each individual state $s_t^{i,k}$ to determine whether or not the made action $a_t^{i,k}$ is good from a global view and next update the actor. Here, we develop a request-specific state $s_t^{i,k}$ including a global state s_t and request observation o_t^{i,q_k} (that will be given very soon). In the execution phase, the agents (i.e., actors) perform in a fully decentralized manner to make actions and yet the centralized critic is unnecessary.

Global State: We define the global state $s_t \in \mathbb{R}^{|\mathcal{E}^c|\cdot F}$ at time step t by the concatenation of the traffic features regarding all cutting edges $\mathcal{E}^c$ in the road network $\mathcal{G}$. It makes sense to use the concatenated features on cutting edges as the global state s_t, instead of the explicit representation of high-dimensional joint actions and observations by M agents in the asynchronous MSD-SPP setting. First of all, since our graph partitioning scheme aims to minimize the number of cutting edges, we expect that the features of such cutting edges is with low dimensionality. Instead, the joint observations and actions of M region agents are typically with the high-dimensionality of $\sum_{i=1}^{M} |\mathcal{Q}_t^i|(|\mathcal{E}_i^c|F + F')$ and $\sum_{i=1}^{M} |\mathcal{Q}_t^i|$, respectively. This high-dimensional representation contains redundant information, leading to inefficiency in MARL training [44]. Moreover, this dimensionality varies with time step, depending on the number of arrival vehicles at the associated regions. Second, each actor R_i makes an action to change the environment, leading to the traffic changes on cutting edges $\mathcal{E}_i^c$. Thus, by using the low-dimensional global state s_t above to implicitly represent the high-dimensional joint observations and actions of multi-agents, we ensure the stationarity of the MARL training in the asynchronous MSD-SPP settings with higher training efficiency.

Request-Specific State: When an agent R_i takes an action $a_t^{i,k}$ in response to the request $q_t^{i,k}$, we define a request-specific state $s_t^{i,k} = [s_t, o_t^{i,q_k}] \in \mathbb{R}^{|\mathcal{E}^c|F+F'}$. This state $s_t^{i,k}$ can be used as a part of the global state s_t above and also as the request observation o_t^{i,q_k}. The critic then estimates the state value $V(s_t^{i,k})$ and the action advantage $\hat{A}_t^{i,k}$, which are used to update the policy of the actor i. In this way, the critic evaluates the state value from a global view and ensures the cooperation in MARL.

To perform the cooperative routing plan in the asynchronous setting, we develop three components on top of the multi-agent actor-critic framework. (1) To address the non-stationary issue of MARL training in the asynchronous MSD-SPP setting, we design an *asynchronous trajectory collection mechanism* (Section IV-A) in the critic network. That is, with the help of the developed global state above, we can asynchronously collect the trajectory of each agent, by inserting the transition of the agent into its separate data buffer immediately, after it takes an action. Different from the synchronous trajectory collection manner, this method does not require that all agents take actions at each time step, and thus mitigates the redundancy issue in the training trajectories. (2) To achieve vehicle cooperation, we develop a novel *actor network* (Section IV-B). The actor effectively encodes road network conditions and routing requests to generate the probability distribution of choosing cutting edges (i.e., action space). In this way, for those vehicles with the same or close destinations, the actor then assigns a higher probability of choosing alternative cutting edges as actions. Such vehicles thus, with low chance, travel on the same road segments to avoid traffic congestion on these segments. (3) To tackle the infinite loop issue caused by incorrect routing decisions, we exploit a *reachability graph* $\mathcal{G}_l^r$ (Section IV-C) for each vehicle l. This graph eliminates road segments leading to infinite loops, and retains only those potential paths with short travel time. Then, we choose those cutting edges appearing within the reachability graph as valid actions.

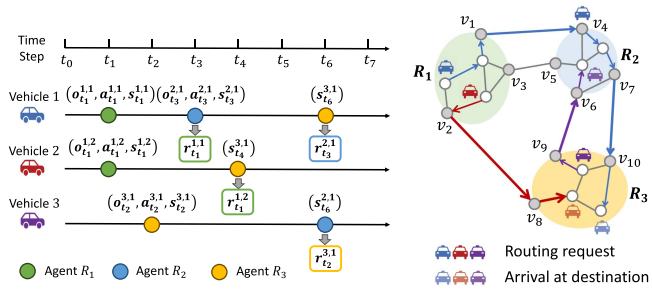
IV. Solution Detail

In this section, we first give the details of the three components (the critic, actor, and reachability graph), and then describe the training details of our aysc-MARL model.

A. Critic Design

The key of our critic network is to develop asynchronous trajectory collection in the asynchronous MSD-SPP setting. We first give the general idea as follows. In the asynchronous MSD-SPP setting, some agents make actions at the time step t and yet others not at this time step. However, once some actions are made, the change of the environment always occurs. Thus, we exploit the aforementioned global state to learn such a change and perform asynchronous trajectory collection as follows.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.



(a) Planning Actions and Traj. Collection (b) Planning Paths

Fig. 4. Example for asynchronous trajectory collection.

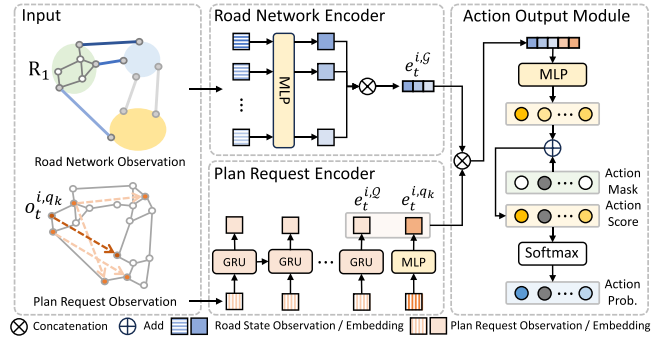


Fig. 5. Actor network structure.

Specifically, whenever an action is made, an agent R_i inserts its own trajectory data into a separate buffer $\mathcal{D}_i$. Denote the transition of a region agent R_i as $(o_t^{i,k}, a_t^{i,k}, s_t^{i,k}, r_t^{i,k}, s_{t'}^{i',k'})$, i.e., given the observation $o_t^{i,k}$ at time step t, the agent R_i makes an action $a_t^{i,k}$ and changes the request-specific state to $s_{t'}^{i',k'}$ at time step t' with the reward $r_t^{i,k}$.

Fig. 4 gives an example of the asynchronous trajectory collection. At time step t_1, vehicles l_1 and l_2 start their travel from region R_1. The region agent R_1 makes two routing actions $a_{t_1}^{1,1}$ and $a_{t_1}^{1,2}$, respectively. Yet the agent R_2 and R_3 do not need to make an action at time step t_1. Here, the critic constructs the global state s_{t_1} at time step t_1 and two request-specific states $s_{t_1}^{1,1} = [s_{t_1}, o_{t_1}^{1,q_1}]$ and $s_{t_1}^{1,2} = [s_{t_1}, o_{t_1}^{1,q_2}]$ when the agent R_1 makes actions $a_{t_1}^{1,1}$ and $a_{t_1}^{1,2}$ for the two vehicles l_1 and l_2, respectively. Next, we compute the reward of the action $a_{t_1}^{1,1}$ as $r_{t_1}^{1,1} = -(t_3 - t_1)$. Now we can insert the transition $(o_{t_1}^{1,1}, a_{t_1}^{1,1}, s_{t_1}^{1,1}, r_{t_1}^{1,1}, s_{t_3}^{2,1})$ into the buffer of agent R_1. After that, when the vehicle l_1 arrives at node v_4 at time step t_3, the agent R_2 selects the edge $v_7 \rightarrow v_{10}$ as the action based on the local observation $o_{t_3}^{2,1}$. Again, the critic constructs the global state s_{t_3} and the request-specific state $s_{t_3}^{2,1} = [s_{t_3}, o_{t_3}^{2,q_1}]$ when agent R_2 makes the action $a_{t_3}^{2,1}$. When the vehicle l_1 finally arrives at its destination in region R_3 at time step t_6, the critic again computes the global state s_{t_6}, request-specific state $s_{t_6}^{3,1}$, and the reward of its previous action $a_{t_3}^{2,1}$ as $r_{t_3}^{2,1} = -(t_6 - t_3)$. Similar to L_1, we collect the following transition trajectories of L_2 and L_3.

$$\begin{aligned}
&\mathcal{D}_1 : [(o_{t_1}^{1,1}, a_{t_1}^{1,1}, s_{t_1}^{1,1}, r_{t_1}^{1,1}, s_{t_3}^{2,1}), (o_{t_1}^{1,2}, a_{t_1}^{1,2}, s_{t_1}^{1,2}, r_{t_2}^{1,2}, s_{t_4}^{3,1})]\\
&\mathcal{D}_2 : [(o_{t_3}^{2,1}, a_{t_3}^{2,1}, s_{t_3}^{2,1}, r_{t_3}^{2,1}, s_{t_6}^{3,1})]\\
&\mathcal{D}_3 : [(o_{t_2}^{3,1}, a_{t_2}^{3,1}, s_{t_2}^{3,1}, r_{t_2}^{3,1}, s_{t_6}^{2,1})]
\end{aligned}$$

After collecting the trajectory data of each agent, we can apply the CTDE paradigm to train a standard MARL model on the collected data. That is, by using the global state s_t and the request observation o_t^{i,q_k} as the request-specific state $s_t^{i,k}$ of agent R_i, the centralized critic exploits an MLP network to estimate the value of $s_t^{i,k}$, i.e., $V(s_t^{i,k}) = \text{MLP}([s_t, o_t^{i,q_k}])$. In this way, with the value $V(s_t^{i,k})$, the critic can estimate the advantage of the action $a_t^{i,k}$ (i.e., $\hat{A}_t^{i,k}$) to update the policy of the local actor i.

B. Actor Design

We propose a novel actor network to facilitate vehicle cooperation and avoid congestion caused by planning the same road for those vehicles with proximate destinations. As show in Fig. 5, the actor consists of an encoder of the road network, an encoder of plan requests, and an output module.

(1) The *road network encoder* learns the traffic features of the road network. Recall that the road network observation $o_t^{i,\mathcal{G}}$ involves the number $|E_i^c|$ of F-dimensional feature vectors regarding the cutting edges $\mathcal{E}_i^c$. Thus, for each cutting edge $b_j \in \mathcal{E}_i^c$, we denote its feature vector by $(o_t^{i,\mathcal{G}})_j$, and feed this vector into multi-layer perceptrons (MLP) to learn a latent embedding vector e_t^{i,b_j} for the edge b_j. By the concatenation of $|\mathcal{E}_i^c|$ vectors for the cutting edges $\mathcal{E}_i^c$, we have the road network state embedding $e_t^{i,\mathcal{G}}$.

$$e_t^{i,\mathcal{G}} = [e_t^{i,b_1}, \ldots, e_t^{i,b_{|\mathcal{E}_i^c|}}], \text{where } e_t^{i,b_j} = \text{MLP}\left((o_t^{i,\mathcal{G}})_j\right) \quad (2)$$

where $[\cdot,\cdot]$ is the concatenation operation and $e_t^{i,b_j} \in \mathbb{R}^{D_h}$, $e_t^{i,\mathcal{G}} \in \mathbb{R}^{|\mathcal{E}_i^c| \times D_h}$, and D_h is the dimensionality of the embeddings.

(2) The *plan request encoder* processes each request observation vector o_t^{i,q_k} through an MLP to learn the embedding $e_t^{i,q_k} \in \mathbb{R}^{D_h}$ for the request $q_t^{i,k}$. To enable the cooperation among those plan requests $\mathcal{Q}_t^i$ within the region R_i, we learn the embeddings of such entire plan requests. Since the number $|\mathcal{Q}_t^i|$ differs from the time step t, we employ a Gated Recurrent Unit (GRU) [5] cell to represent such requests $\mathcal{Q}_t^i$ as follows.

$$h_{k'} = \text{GRU}(o_t^{i,q_{k'}}, h_{k'-1}), k' = 1, \ldots, |\mathcal{Q}_t^i| \quad (3)$$

where the plan request observation vector $o_t^{i,q_{k'}} \in \mathbb{R}^{F'}$ and $h_{k'} \in \mathbb{R}^{D_h}$ are input vector and the hidden state of the GRU cell at step k', respectively. Given the sequential GRU model, we take the hidden state of the final GRU cell as the embedding of $\mathcal{Q}_t^i$, i.e., $e_t^{i,\mathcal{Q}} = h_{|\mathcal{Q}_t^i|} \in \mathbb{R}^{D_h}$.

(3) Now we have three embeddings, i.e., $e_t^{i,\mathcal{G}}$, e_t^{i,q_k}, and $e_t^{i,\mathcal{Q}}$. With such embeddings as input, the actor generates a probability distribution on the cutting edges $\mathcal{E}_i^c$ and chooses a certain cutting edge with the greatest probability. Specifically, the output module first maps the embeddings to a score vector $u_t^{i,k} \in \mathbb{R}^{|\mathcal{E}_i^c|}$ on cutting edges $\mathcal{E}_i^c$ with an MLP. Then, we apply

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

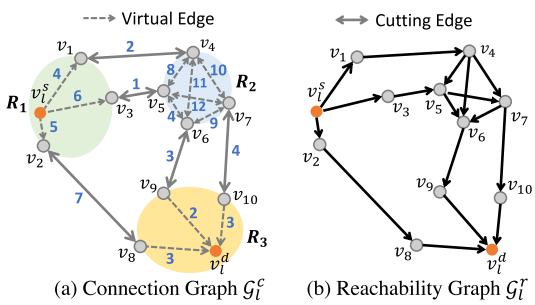


Fig. 6. An example of reachability graph construction .

the invalid action mask [14] with the help of the reachability graph (that will be soon given in Section IV-C) to avoid loops in the solution paths. In this way, we compute the score of a cutting edge b_j by $(u_t^{i,k})_j = (\text{MLP}([e_t^{i,\mathcal{G}}, e_t^{i,q_k}, e_t^{i,\mathcal{Q}}]))_j$ if the chosen action b_j is within the reachability graph and otherwise $(u_t^{i,k})_j = -\infty$. Finally, we employ a softmax function [3] to transform the edge scores into a probability distribution on $\mathcal{E}_i^c$.

$$P(a_t^{i,k} = b_j) = \frac{\exp((u_t^{i,k})_j)}{\sum_{j'=1}^{|\mathcal{E}_i^c|} \exp((u_t^{i,k})_{j'})} \tag{4}$$

Here, a greater edge score corresponds to a higher probability, and the probability to select invalid edge becomes zero. The intuition of the probability distribution is as follows. With help of the embeddings e_t^{i,q_k} and $e_t^{i,\mathcal{Q}}$, the actor can identify those requests towards the same or close destinations. If the number of such requests is small, the actor tends to assign a high probability (near 1.0) to the cutting edge which leads to the shortest travel time for the request $q_t^{i,k}$. Otherwise, the actor assigns an even probability distribution across the cutting edges $\mathcal{E}_i^c$. In this way, we have chance to avoid planning too many vehicles on the same road segments and thus achieve cooperation among the vehicles.

C. Reachability Graph

Due to the lack of a global view, the inter-region routing plan may suffer from the issue of infinite loops. To overcome this issue, we construct a *reachability graph* for each vehicle $l = \langle v_l^s, v_l^d, t_l^s \rangle$. First, when given a vehicle l starting from the source v_l^s to the destination v_l^d, the source region agent processes the plan request of l and constructs the reachability graph $\mathcal{G}_l^r$ via the underlying static road network and observed traffic conditions. It makes sense for the agent R_i to construct $\mathcal{G}_l^r$: every agent R_i can observe the static information of the entire road network and the estimation information of neighboring regions. For the road segments outside the region, the agent R_i can estimate their travel time based on the road lengths and the average speed of the located regions.

To construct a reachability graph $\mathcal{G}_l^r$, the source region first needs to build a *connection graph* $\mathcal{G}_l^c$ to represent the potential paths from the source v_l^s to destination v_l^d intermediately through boundary nodes and cutting edges. As shown in Fig. 6(a), given the source v_l^s and destination v_l^d, we construct three virtual edges from source node v_l^s to the boundary nodes in source region R_1, $v_l^s \to v_1$, $v_l^s \to v_2$ and $v_l^s \to v_3$. Again, we construct virtual edges from boundary nodes in destination region R_3 to the destination node v_l^d. Moreover, we construct virtual edges among boundary nodes in the intermediately connected regions. For a virtual edge inside the current region R_1 (e.g., $v_l^s \to v_1$), the region agent assigns an edge weight based on the least travel time on the current region. This is feasible, since the agent can fully observe the detailed travel time of all roads within the region. For the virtual edges in other regions, the agent estimates the weights by dividing the shortest path distances on the static road network by the observed average speed of the corresponding region.

Now, we can build a region-level a path based on the connection graph $\mathcal{G}_l^c$. For instance, the path $v_l^s \to v_1 \to v_4 \to v_7 \to v_{10} \to v_l^d$ indicates that the vehicle starts from region R_1, enters the region R_2 with the chosen cutting edge $v_1 \to v_4$, and reaches the destination region R_3 with the final cutting edge $v_7 \to v_{10}$. Note that the connection graph $\mathcal{G}_l^c$ may involve many loops, for example, $v_4 \to v_6 \to v_7 \to v_5 \to v_4$. To avoid loops during route planning, we convert the connection graph $\mathcal{G}_l^c$ into the reachability graph $\mathcal{G}_l^r$ by a directed acyclic graph (DAG) conversion algorithm [13] with the purpose to prune those edges that lead to loops and retain relatively short paths as many as possible. Due to space limit, we refer interested readers to the paper [13]. On the connection graph $\mathcal{G}_l^c$ in Fig. 6(a), we give the reachability graph $\mathcal{G}_l^r$ in Fig. 6(b) after the conversion. The complexity of the reachability graph construction is $\mathcal{O}(|\mathcal{E}_l^c| + |\mathcal{V}_l^c| \log |\mathcal{V}_l^c|)$, where $|\mathcal{V}_l^c|$ and $|\mathcal{E}_l^c|$ are the numbers of nodes and edges in connection graph $\mathcal{G}_l^c$, respectively. Since the size of the connection graph $\mathcal{G}_l^c$ is much smaller than the original road network graph, the construction cost of $\mathcal{G}_l^r$ is trivial.

When the vehicle l arrives at a new region R_i, the agent updates its reachability graph based on the latest traffic observations. Specifically, the agent R_i updates the edge weights in the connection graph $\mathcal{G}_l^c$ and converts it to the reachability graph $\mathcal{G}_l^r$ with the DAG conversion. Then, the region agent R_i will mask the invalid action roads that are infeasible in the graph $\mathcal{G}_l^r$ when making inter-region route planning. For instance, the reachability graph $\mathcal{G}_l^r$ indicates that agent R_2 receives the request from the vehicle l, and the valid actions for its request are confined to $\{v_6 \to v_9, v_7 \to v_{10}\}$.

D. Training

Given the trajectory collection in Section IV-A, we exploit Multi-agent Proximal Policy Gradient (MAPPO) [44] to train the actor-critic network on the data buffers of M agents. Algorithm 1 gives the training steps of asyn-MARL. Line 1 initializes the network parameters of M actors and the critic, i.e., θ_i with $1 \leq i \leq M$ and ϕ randomly. Next, with the **for** loop of each episode, we first empty the data buffers of all actors (line 3). Then, at each time step, the critic accesses the global state s_t (line 5). Each actor receives a set of routing requests (line 7), and makes actions with its policy π^i parameterized with θ_i for the observation $o_t^{i,k}$ (line 9). Next, the actor receives the reward $r_t^{i,k}$ and inserts the transition into its data buffer (line 10). Each transition consists of the local observation $o_t^{i,k}$, action $a_t^{i,k}$, reward $r_t^{i,k}$, state $s_t^{i,k}$, and the next state $s_{t'}^{i',k'}$. After that, lines 11 - 12 compute the

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Train the Actor-Critic Net. of asyn-MARL.

1 Initialize the params. of M actors θ_i and centralized critic ϕ;
2 **for** *episode =1 to* E **do**
3 Set data buffer $\mathcal{D}_i = \{\}$ for $i = 1 \ldots M$;
4 **for** *time step* $t \in 1 \ldots T$ **do**
5 Get the global network state s_t;
6 **for** $1 \leq i \leq M$ **do**
7 Get the plan request set $\mathcal{Q}_t^i$ of actor θ_i;
8 **for** *request* $q_t^{i,k} \in \mathcal{Q}_t^i$ **do**
9 R_i makes action $a_t^{i,k}$ on obs. $o_t^{i,k}$ with π^i;
10 Insert $(o_t^{i,k}, a_t^{i,k}, s_t^{i,k}, r_t^{i,k}, s_{t'}^{i',k'})$ into $\mathcal{D}_i$;
11 Compute advantage $\hat{A}_t^{i,k}$ of $a_t^{i,k}$ by Eq. 5;
12 Compute discounted return $\hat{G}_t^{i,k}$ by Eq. 6;
13 **end**
14 **end**
15 **end**
16 **for** $i = 1, \ldots, M$ **do**
17 Update the actor parameter θ_i by Eq. 7 on $\mathcal{D}_i$;
18 **end**
19 $\mathcal{D} = \bigcup_{i=1}^{M} \mathcal{D}_i$;
20 Update the critic parameter ϕ by Eq. 8 on $\mathcal{D}$;
21 **end**

advantage $\hat{A}_t^{i,k}$ by (5) and discounted return $\hat{G_t^{i,k}}$ by (6).

$$\hat{A}_t^{i,k} = r_t^{i,k} + \gamma V(s_{t'}^{i',k'}) - V(s_t^{i,k}) \tag{5}$$

where $\gamma \in (0, 1]$ is the discount factor and V is the global value function of the centralized critic (here, the value function of the critic is parameterized with ϕ).

$$\hat{G_t^{i,k}} = r_t^{i,k} + \gamma V(s_{t'}^{i',k'}) \tag{6}$$

where $r_t^{i,k}$ is the reward for action $a_t^{i,k}$ and $V(s_{t'}^{i',k'})$ is the value of the new state $s_{t'}^{i',k'}$ after the action $a_t^{i,k}$ is made.

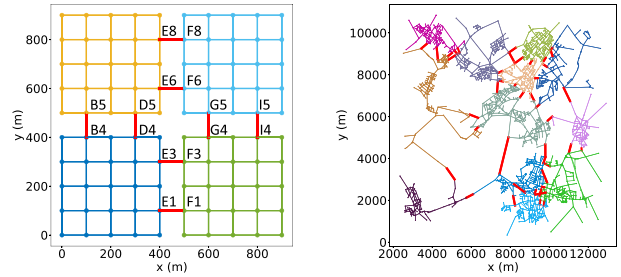
After an episode, lines 16–18 then update the policy parameters of every actor θ_i. By the PPO-Clip algorithm [28], we update the actor policy by maximizing the surrogate objective.

$$L(\theta_i) = \mathbb{E}_{o_t^{i,k}, a_t^{i,k}}[\min(\rho_t^{i,k}\hat{A}_t^{i,k}, \text{clip}(\rho_t^{i,k}, 1-\epsilon, 1+\epsilon)\hat{A}_t^{i,k})] \tag{7}$$

where $\hat{A}_t^{i,k}$ is computed by (5) above, and $\rho_t^{i,k} = \frac{\pi^i(a_t^{i,k}|o_t^{i,k})}{\pi_{old}^i(a_t^{i,k}|o_t^{i,k})}$ is the ratio between the current policy π^i and the old one π_{old}^i that is used to sample the trajectory by the actor θ_i. This ratio is clipped to $[1-\epsilon, 1+\epsilon]$ to avoid a significant policy update and $\epsilon \in (0, 1)$ is the PPO clipping parameter.

Finally, lines 19–20 combine the M data buffers to an entire buffer $\mathcal{D}$ and update the critic parameters ϕ by minimizing the loss $L(\phi)$, e.g., a mean-squared error, on the data buffer $\mathcal{D}$. Here, the discounted return $\hat{G_t^{i,k}}$ is given by (6).

$$L(\phi) = \mathbb{E}_{s_t^{i,k}}\left(V(s_t^{i,k}) - \hat{G}_t^{i,k}\right)^2 \tag{8}$$



(a) Synthetic (100 nodes) (b) Koln (2515 nodes)

Fig. 7. Road network data sets (Cutting edges are plotted as red lines.).

V. Experiment

A. Data Sets

We conduct experiments on two road networks (one synthetic and one real-world), and perform traffic simulation on a widely used simulator, Simulation of Urban Mobility (SUMO) [22].

1) Synthetic Data Set: Fig. 7(a) illustrates the synthetic road network of 4 regions. Each region consists of 25 intersections, 80 internal road segments. The length of road segments is 100 meters. Since each region involves 4 cutting edges (i.e., the action space is 4), this synthetic road network involves totally 100 vertices and 336 edges (i.e., 320 internal and 16 cutting edges). We generate traffic flows from the yellow region to the green one, and then randomly choose source and destination intersection nodes within the two regions, respectively. Moreover, we set the maximum speed on each road segment as 13.89 m/s.

2) Real Data Set: We use a traffic data set provided by the previous work [35], which provides the road network data in Koln, Germany. In this data set, we select an area of 100 km^2, from $6.166°E, 50.735°N$ to $7.295°E, 51.840°N$. The selected area contains 2515 intersections and 5784 road segments. Following the data pre-processing technique [1], we remove the isolated road segments and dead-ends and then employ METIS [18] to divide the road network into 12 regions with similar sizes and totally 84 cutting edges. As shown in Fig. 7(b), the number of cutting edges per a region ranges from 2 to 11 with an average 7. The road network involves 13 types of road segments. Depending upon the road types, we set the maximum travel speed of each road segment. For example, the maximum speeds of the primary road and secondary road are 19.44m/s and 13.89m/s, respectively.

B. Traffic Simulation

During the simulation, we inject vehicles into a road network every second until the total number of injected vehicles reaches a predefined value (e.g., 200 and 1500 for the synthetic and real road networks, respectively). When the number of vehicles on a road segment exceeds the road capacity limit, we follow the previous work [2] to simulate traffic congestion by decreasing the maximum speed by αV_{max}. Here, the factor $\alpha = 0.1$ and V_{max} is the original maximum speed of the road segment.

By default, on the synthetic network, we set the capacity limits, i.e., the number of vehicles on the road, as 10, and

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS FOR THE TWO DATA SETS

Data Set	Synthetic	Koln
num. of Episode Steps T	600	600
num. of Maximum Vehicles in Networks	200	1500
Road Capacity Limit	10	[5, 272]
Average Road Maximum Speed (m/s)	13.89	11.42

TABLE III
PARAMETER CONFIGURATION FOR ASYN-MARL

Parameter	Value	Parameter	Value
dim. of embeddings D_h	32	num. of mini-batches	1
PPO clipping parameter ϵ	0.2	num. of PPO epochs	15
discount factor γ	0.99	actor learning rate	10^{-5}
maximum episode E	200	critic learning rate	10^{-5}

Section V-D5 will evaluate the performance of asyn-MARL under various capacity limits and maximum traffic volumes on the synthetic network. On Koln road network, we set the capacity constraint of a road within the range from 5 to 272. Specifically, following the recommended setting in SUMO [22], each vehicle length is 5 meters and the minimal space gap between neighboring vehicles is 2.5 meters. Then, each road e_{ij} with the road length h_{ij} can accommodate at most $\left\lceil \frac{h_{ij}}{5+2.5} \right\rceil$ vehicles. In this way, we follow the road length in the Koln dataset to set the aforementioned range [5, 272]. In addition, we set the simulation episode as 600 seconds for the two data sets. Tables II and III gives the simulation parameters on the two data sets and the hyper-parameters of the RL model.

C. Baselines and Metrics

We compare our asyn-MARL with three traditional shortest path-based and two MARL-based algorithms.

- *Random:* The agents perform the inter-region routing plan by randomly choosing a cutting edge to neighbour regions.
- *Shortest Path (SP):* The agents select a cutting edge which appears on the shortest path to the destination based on static road networks.
- *Shortest Path First with Re-routing (SPFR):* Following the previous work [2], we pre-compute shortest paths on static road networks, and then update the route based on current traffic observation when a vehicle enters a new road segment. For fairness, we follow the same assumption as ours to ensure that this algorithm works in our setting: an agent can observe the detailed traffics on the road segments within the associated region, and roughly estimate the traffics of neighbour regions.
- *Adaptive navigation (AN):* The work [2] models each intersection as an agent to select the next-hop intersection for the coming vehicles. This work utilizes a multi-agent deep Q-Network model to train intersection agents, which involves the graph attention network [38] to learn the traffic on neighbour road segments of the intersection.
- *Q-routing:* To be consistent with [2], we choose Q-Routing [4] as our baseline. Though originally developed for network packet routing, Q-Routing [4] and our work share the very similar task to find the shortest path route. It estimates a Q-function $Q_x(d, y)$ with a large table to represent the time to arrive at node d from a node x by the way of x's neighbor node y. We use the MARL implementation of Q-Routing provided by [2] to estimate the Q-function. This implementation treats each intersection as an agent and learns the values of selecting its neighbor nodes for a given destination node. This baseline is not aware of the global road network state.

We use three metrics for performance evaluation.

- *Throughput (TPT):* the total number of vehicles which can successfully reach their destinations within an episode.
- *Average travel time (AVTT) (seconds):* For those vehicles which successfully reach their destinations within an episode, we compute the average travel time of such vehicles from the sources to destinations.
- *Average CO2 emission volume (CO2):* We compute the average CO2 emission volume (kg) of each vehicle by following the CO2 emission model HBEFA.[1]

TABLE IV
BASELINE STUDY

	Sythetic Data Set			Koln Data Set		
	TPT	AVTT	CO2	TPT	AVTT	CO2
Random	320.30	313.503	0.807	164.20	1295.366	2.988
SP	738.02	158.837	0.416	513.60	1030.955	2.246
SPFR	784.34	150.864	0.384	530.40	997.142	2.227
Q-Routing	222.48	413.444	1.089	/	/	/
AN	764.15	168.967	0.444	/	/	/
asyn-MARL	936.94	125.551	0.352	568.3	978.836	2.195

Results of Q-Routing and an on Koln Data Set are unavailable because they cannot scale to the large graphs.

D. Experiment Results

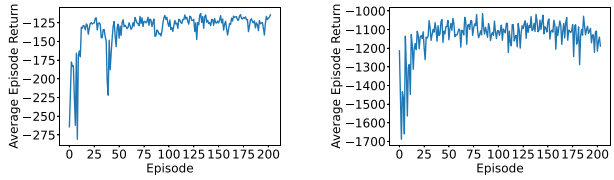
1) Baseline Study: Table IV gives the baseline result of our work (asyn-MARL) and five counterparts. From this table, we have the following findings. On the two road networks, our work outperforms all five counterparts on the three metrics. First, the two algorithms, random and SP, compute the static shortest paths and cannot adapt to the dynamic traffics caused by the competition of multiple routing plans. Thus, some road segments are with too many vehicles, leading to traffic congestion and long travel time. For example, the average travel time of SP is 26% longer than asyn-MARL.

Second, SPFR updates the pre-computed shortest paths based on current road conditions and outperforms the Random and SP algorithms. However, it cannot offer vehicle cooperation and still leads to worse result than ours.

Third, the Q-Routing and AN algorithms exploit MARL to select next-hop intersections as actions. Here, since Q-routing is not aware of the global network state, it frequently makes incorrect decisions, leading to infinite loops. The infinite loops issue becomes greatly severe on large road networks. Thus, some vehicles cannot reach their destinations, resulting in significantly longer travel time. In addition, AN leads to better performance

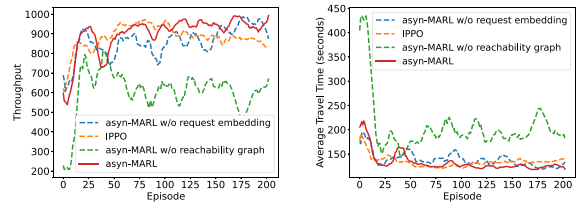
[1] www.hbefa.net

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.



(a) Synthetic (b) Koln

Fig. 8. Curve of episode rewards.



(a) Throughput (b) Average Travel Time (second)

Fig. 9. Results of ablation study.

than Q-Routing, due to the adopted graph attention network to incorporates the information of neighbour road segments.

When comparing the results on two road networks, we find that the performance in the Koln data is worse than the one in the synthetic data, e.g., lower throughput and higher average travel time (AVTT) and CO2 emission. The main reason is that the area of the Koln network is almost 10 times greater than the synthetic data set, resulting in considerably longer travel time and CO2 emission volumes for vehicles to reach destinations. In terms of throughput, the Koln data set has more complex road segments than the synthetic data set, the associated average travel speed is much slower, and more travel requests are not finished within an episode of the simulation, indicating smaller throughput on the Koln network. Moreover, the Koln data is with 2515 intersections. It is rather hard to train a very large MARL-based planning model with 2515 agents in Q-Routing and AN. Such result alternatively demonstrates the efficiency of our work asyn-MARL.

In addition, Fig. 8 gives the average reward per episode during training. Our work asyn-MARL converges within 200 episodes on the two data sets. Notably, asyn-MARL converges faster and exhibits smaller fluctuations in episode rewards on the synthetic road network than the result on Koln data set, because the synthetic road network is with rather regular topology structure.

2) Ablation Study: To study the benefits of the three components, i.e., the actor network, the centralized critic and the reachability graph, we conduct ablation study and evaluate the performance of asyn-MARL with three following variants.

- *asyn-MARL w/o request embedding:* we remove the request encoding of other requests and output the action selection with simple MLP network directly.
- *IPPO:* we train the multi-agents independently with the PPO algorithm [28]. Unlike our work asyn-MARL, this variant does not have a centralized critic, and instead each agent has a local version of the actor and critic by using the local observation as input yet without considering the joint observations and actions of other agents (i.e., the global state s_t in asyn-MARL).
- *asyn-MARL w/o reachability graph:* we remove the reachability graph and the associated action masks during action decision.

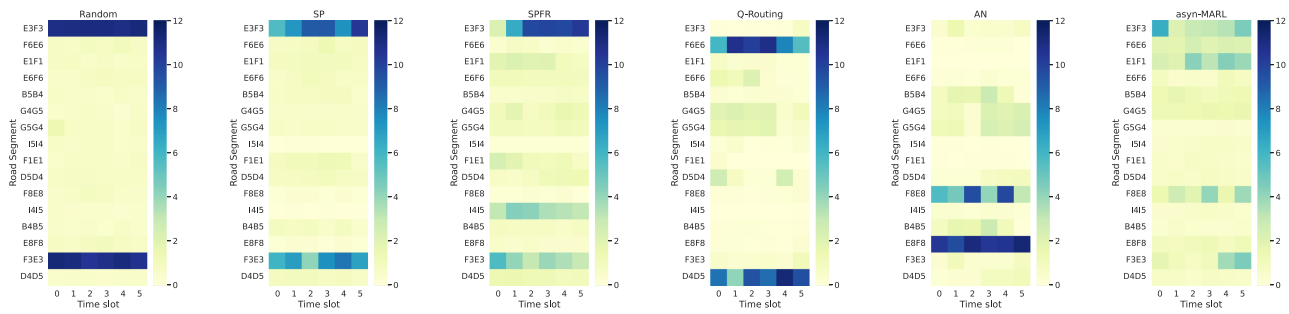
As shown in Fig. 9, we plot the throughput and average travel time of the four algorithms during the training phase. Compared to our work asyn-MARL, the three variants lead to lower throughput and greater AVTT. In particular, the variant without reachability graphs suffers from the worst performance. From this ablation study, the evaluation results indicate the importance of the three components in our framework aysn-MARL.

3) Load Balance Study: In this section, we evaluate the load balance of all algorithms in terms of the distribution of average traffic volumes (the number of vehicles) on the entire cutting edges. As shown in Fig. 10, the x-axis and y-axis indicate the time slot during a span of the simulation with 5 minutes and the ID number of 16 cutting edges, which are marked in Fig. 7(a). and the color darkness instead means the traffic volumes on the associated cutting edge within the time slot. As shown in this figure, our asyn-MARL demonstrates a rather even distribution of traffic volumes on the 16 cutting edges across the entire time slots. Instead. the SP and Random approaches tend to plan too many vehicles towards two cutting edges (E3F3 and F3E3), resulting in traffic congestion. Instead, SPFR could re-route some traffics towards other cutting edges such as I4I5, and lead to shorter travel time. In addition, though both Q-Routing and AN can re-route some traffics from the two cutting edges (E3F3 and F3E3) to alternative edges, such re-routed traffics are unfortunately re-directed to the same cutting edges, i.e., D4D5 in Fig. 9(d) and F8E8 in Fig. 9(e), again suffering from traffic congestion. In contrast, due to the proposed actor and critic networks, asyn-MARL has the chance to choose diverse cutting edges even for those vehicles with close destinations, and thereby mitigates traffic congestion for better load balancing.

4) Effect of Graph Division: In this section, we study the effects of graph division on the synthetic data set. By varying the number of regions from 3 to 6, we compare the effect of two graph division algorithms. (1) Random Approach: we randomly select M nodes as the initial seed nodes of the M regions, then iteratively add a neighbour node to each region until all the nodes are assigned to the associated regions. (2) We use METIS [18] to divide the road graph. This algorithm aims to create sub-graphs while minimizing the number of cutting edges and ensuring sub-graphs with similar size. Fig. 7 colorizes the divided sub-graphs (regions) and we comfortably recognize that such division is roughly consistent with the overall graph topology.

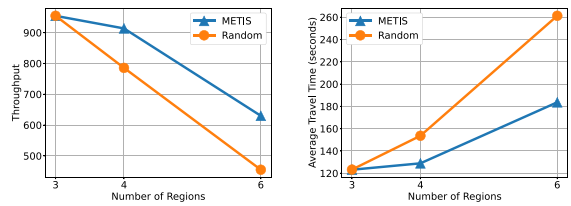
Fig. 11 gives the performance of two graph division approaches. We can observe that random division yields worse performance. That is, the random division generates more cutting edges and higher action space of region agents than METIS. For instance, when the number of regions is 4, the numbers of cutting edges generated by METIS and random division are 16 and 62, respectively. Thus, it makes sense that the asyn-MARL model trained on the randomly divided regions suffers from

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.



(a) Random (b) SP (c) SPFR (d) Q-Routing (e) AN (f) asyn-MARL

Fig. 10. Distribution of vehicle volume on cutting edges with different routing methods on synthetic data set.



(a) Throughput (b) Average Travel Time (second)

Fig. 11. Effect of graph division approaches (Random and METIS).

worse result. Moreover, when the number of regions grows, the performance of asyn-MARL decreases, due to the following reason. Given a smaller number of regions, we have make the implicit assumption that a region agent has the higher capacity to observe the details of the road segments within in a larger region. Otherwise, when the number of subgraphs is greater, region agents can observe the details of a smaller region and yet have to roughly estimate the majority of the entire graph, leading to performance degrade.

5) Effect of Road Capacity Limit and Traffic Volume: We finally evaluate the performance of our work under different traffic settings. Fig. 12(a)–(c) gives the performance of the six methods by varying the road capacity limits $c \in \{8, 10, 12\}$. We can find that when the capacity limit is smaller, i.e., the road segments are more likely to be congested, our work asyn-MARL consistently outperforms the competitors. Note that, for a small capacity limit such as 8, the performance gap of our work and the competitor is significant. Such result indicates that our work asyn-MARL prefers to work well in the setting with limited resource.

Fig. 12(d)–(f) studies the performance of asyn-MARL when the vehicle count in the synthetic road network varies from 150 to 250. We can find that, when the vehicle count is smaller than 200, the performance of SP, SPFR and asyn-MARL is relatively close, because the road segments are less likely to be congested. As the vehicle count grows, the competition becomes more severe, and the average travel time of all approaches grows. In particular, in such a setting with more resource competition (e.g., caused by more vehiles on the road network), our work asyn-MARL outperforms others. The results again indicate that our work asyn-MARL works well in the setting.

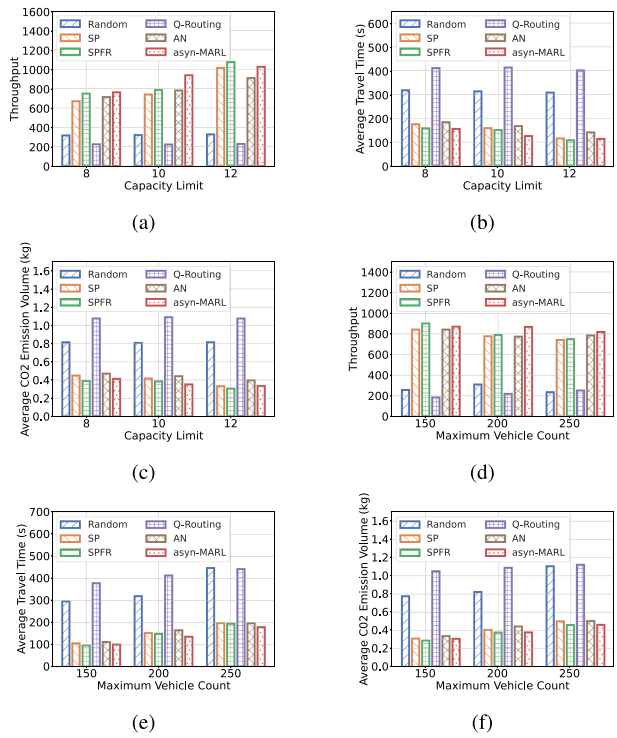
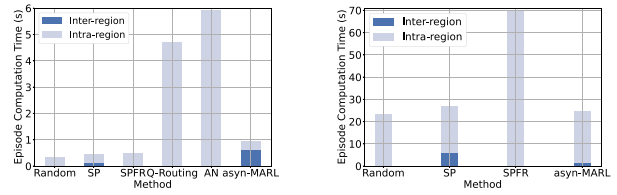


Fig. 12. Effect of road capacity (a), (b), and (c) and traffic volume (d), (e), (f).



(a) Synthetic (100 nodes) (b) Koln (2515 nodes)

Fig. 13. Computation time on two datasets (results of Q-routing and AN on the Koln dataset are unavailable because they cannot scale to the large graphs).

6) Efficiency Study: We evaluate the efficiency of all the methods on a machine with 48 GB RAM, one 2.50 GHz CPU, and one RTX 2080 Ti GPU for forward action computation and training updates. Fig. 13 plots the average computation time per episode. Among the five competitors, SPFR, Q-Routing, and AN do not require the two-stage route planning, and directly perform

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

the next-hop node-based planning. For fairness, we employ the same intra-region route planning for the three schemes including Random, SP and asyn-MARL. Thus, such three schemes are with the roughly similar intra-region computation time. Note that the time ratio of the intra-region route planning is higher, mainly because we update the intra-region route whenever vehicles arrive at intersections.

On the synthetic dataset, Q-Routing and AN suffer from the most computation time: whenever a vehicle arrives at a new intersection, the two MARL algorithms have to learn node embeddings on the entire road graph and then choose a next-hop intersection. On Koln dataset, the computation time of Random, SP and asyn-MARL is less than that of SPFR. It is because the three methods solve the intra-region routing plans on divided regions, much smaller than the entire road network. Moreover, asyn-MARL slightly outperforms SP in terms of the inter-region computation time on this larger road network due to two reasons. (1) The computation time for asyn-MARL to solve a single routing request (1.76 ms) is shorter than that of SP (3.44 ms) because the computation time of SP grows much more than asyn-MARL on a greater graph, and (2) asyn-MARL can solve multiple routing requests in parallel by multi-GPUs on the Koln dataset.

E. Discussion

Fine-grained traffic information: In this paper, we make routing plans via such road segment observations as the associated road length, locations and traffic volume and speed. We do not explicitly use the observation data on intersections, e.g., the length of waiting queues. Nevertheless, the observations on road segments are implicitly correlated to the observation on intersections. Moreover, we believe that our developed asyn-MARL model can comfortably incorporate more intersection observations e.g., the length of waiting queues and even the status of traffic lights, for better performance.

Dynamic road network partitioning: We partition the road network into non-overlapping sub-graphs (i.e., regions) based on its static topology structure. However, due to the dynamic spatio-temporal distribution of traffic flow [6], there may exist significant differences regarding the numbers of plan requests received by different region agents, as well as the numbers of plan requests at different time periods. One possible solution is to partition the road network based on such dynamic traffic flows for more balanced workload of region agents.

VI. Conclusion

In this paper, we propose a cooperative shortest path planning framework in the asynchronous MSD-SPP setting. To deal with the ineffectiveness and inefficiency issue of existing works, we propose the two-stage routing plans (inter-region and intra-region plans), and formulate the inter-region planning as a decentralized partial observable Markov decision process (Dec-POMDP). The proposed framework asyn-MARL is an actor-critic-based algorithm consisting of a centralized critic improved by asynchronous trajectory collection to address the non-stationary issue of MARL training, a novel actor to cooperate the vehicles with close destinations, and a reachability graph to avoid infinite loops. On both synthetic and real datasets, our evaluation demonstrates that our work asyn-MARL outperforms both the classic shortest path computation algorithms and the recent MARL-based baselines.

References

[1] M. S. Ahmed and M. A. Hoque, "Partitioning of urban transportation networks utilizing real-world traffic parameters for distributed simulation in sumo," in *Proc. 2016 IEEE Veh. Netw. Conf.*, 2016, pp. 1–4.

[2] F. Arasteh, S. SheikhGarGar, and M. Papagelis, "Network-aware multi-agent reinforcement learning for the vehicle navigation problem," in *Proc. 30th Int. Conf. Adv. Geographic Inf. Syst.*, 2022, pp. 69:1–69:4.

[3] C. M. Bishop and N. M. Nasrabadi, *Pattern Recognition and Machine Learning*, vol. 4. Berlin, Germany: Springer, 2006.

[4] J. Boyan and M. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in *Proc. Adv. Neural Inf. Process. Syst.*, 1993, pp. 671–678.

[5] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 1724–1734.

[6] X. Di, Y. Xiao, C. Zhu, Y. Deng, Q. Zhao, and W. Rao, "Traffic congestion prediction by spatiotemporal propagation patterns," in *Proc. 20th IEEE Int. Conf. Mobile Data Manage.*, 2019, pp. 298–303.

[7] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[8] T. Eilam-Tzoreff, "The disjoint shortest paths problem," *Discrete Appl. Math.*, vol. 85, no. 2, pp. 113–138, 1998.

[9] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2974–2982.

[10] C. Gawron, "An iterative algorithm to determine the dynamic user equilibrium in a traffic simulation model," *Int. J. Modern Phys. C*, vol. 9, no. 3, pp. 393–407, 1998.

[11] Y. Geng et al., "Deep reinforcement learning based dynamic route planning for minimizing travel time," in *Proc. 2021 IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, 2021, pp. 1–6.

[12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[13] O. Hope and E. Yoneki, "GDDR: GNN-based data-driven routing," in *Proc. 41st IEEE Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 517–527.

[14] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. 35th Int. Florida Artif. Intell. Res. Soc. Conf.*, 2022, pp. 1–7.

[15] Z. Huang and B. P. Y. Loo, "Urban traffic congestion in twelve large metropolitan cities: A thematic analysis of local news contents, 2009–2018," *Int. J. Sustain. Transp.*, vol. 17, no. 6, pp. 592–614, 2023.

[16] INRIX, "Inrix 2023 global traffic scorecard," 2023. [Online]. Available: https://inrix.com/scorecard/

[17] R. Jaikumar, S. Shiva Nagendra, and R. Sivanandan, "Modal analysis of real-time, real world vehicular exhaust emissions under heterogeneous traffic conditions," *Transp. Res. Part D: Transport Environ.*, vol. 54, pp. 397–409, 2017.

[18] G. Karypis and V. Kumar, "METIS: Unstructured graph partitioning and sparse matrix ordering system," Univ. Minnesota, Minneapolis, MN, USA, Tech. Rep. 97-061, 1997.

[19] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 7166–7190.

[20] Y. Liang, H. Wu, and H. Wang, "ASM-PPO: Asynchronous and scalable multi-agent PPO for cooperative charging," in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst.*, 2022, pp. 798–806.

[21] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. 4th Int. Conf. Learn. Representations*, 2016, pp. 1–14.

[22] P. A. Lopez et al., "Microscopic traffic simulation using sumo," in *Proc. 21st Int. Conf. Intell. Transp. Syst.*, 2018, pp. 2575–2582.

[23] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst. 30: Annu. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6379–6390.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.

[24] F. A. Oliehoek and C. Amato, "A concise introduction to decentralized POMDPs," in *Springer Briefs in Intelligent Systems*. Berlin, Germany: Springer, 2016.
[25] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Appl. Intell.*, vol. 53, no. 11, pp. 13677–13722, 2023.
[26] G. Papoudakis, F. Christianos, A. Rahman, and S. V. Albrecht, "Dealing with non-stationarity in multi-agent deep reinforcement learning," 2019, *arXiv: 1906.04737*.
[27] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. N. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4292–4301.
[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*.
[29] P. K. Sharma, R. Fernandez, E. G. Zaroukian, M. R. Dorothy, A. Basak, and D. E. Asher, "Survey of recent multi-agent reinforcement learning algorithms utilizing centralized training," 2021, *arXiv:2107.14316*.
[30] Z. Shou, X. Chen, Y. Fu, and X. Di, "Multi-agent reinforcement learning for Markov routing games: A new modeling paradigm for dynamic traffic assignment," *Transp. Res. Part C: Emerg. Technol.*, vol. 137, 2022, Art. no. 103560.
[31] K. Son, D. Kim, W. J. Kang, D. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proc. Mach. Learn. Res.*, 2019, pp. 5887–5896.
[32] Q. Sun, L. Zhang, H. Yu, W. Zhang, Y. Mei, and H. Xiong, "Hierarchical reinforcement learning for dynamic autonomous vehicle navigation at intelligent intersections," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 4852–4861.
[33] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning," 2017, *arXiv: 1706.05296*.
[34] A. Tampuu et al., "Multiagent cooperation and competition with deep reinforcement learning," 2015, *arXiv:1511.08779*.
[35] S. Uppoor, O. Trullols-Cruces, M. Fiore, and J. M. Barcelo-Ordinas, "Generation and analysis of a large-scale urban vehicular mobility dataset," *IEEE Trans. Mobile Comput.*, vol. 13, no. 5, pp. 1061–1075, May 2014.
[36] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. 16th ACM Workshop Hot Topics Netw.*, 2017, pp. 185–191.
[37] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[38] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Representations*, 2018, pp. 2920–2931.
[39] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2692–2700.
[40] J. Wang and L. Sun, "Reducing bus bunching with asynchronous multi-agent reinforcement learning," in *Proc. 13th Int. Joint Conf. Artif. Intell.*, 2021, pp. 426–433.
[41] Z. Wang and S. Wang, "XRouting: Explainable vehicle rerouting for urban road congestion avoidance using deep reinforcement learning," in *Proc. IEEE Int. Smart Cities Conf.*, 2022, pp. 1–7.
[42] J. Yin, W. Rao, and C. Zhang, "Learning shortest paths on large dynamic graphs," in *Proc. 22nd IEEE Int. Conf. Mobile Data Manage.*, 2021, pp. 201–208.
[43] J. Yin, W. Rao, Q. Zhao, C. Zhang, and P. Hui, "Learn to optimize the constrained shortest path on large dynamic graphs," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2456–2469, Mar. 2024.
[44] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24611–24624.
[45] C. Yu et al., "Asynchronous multi-agent reinforcement learning for efficient real-time multi-robot cooperative exploration," in *Proc. 2023 Int. Conf. Auton. Agents Multiagent Syst.*, 2023, pp. 1107–1115.
[46] B. Zhu, J. Li, Q. Yuan, J. Lu, and S. Yang, "An edge-assisted vehicle routing method based on game-theoretic multiagent learning," in *Proc. IEEE 25th Int. Conf. Parallel Distrib. Syst.*, 2019, pp. 462–469.

Jiaming Yin received the BE degree from Tongji University, Shanghai, China, in 2019. She is currently working toward the PhD degree with the School of Computer Science and Technology. Her research interests include time series, reinforcement learning and mobile computing.

Weixiong Rao (Member, IEEE) received the PhD degree from the Chinese University of Hong Kong, Hong Kong, in 2009. After that, he worked for Hong Kong University of Science and Technology, in 2010, University of Helsinki (2011-2012), and University of Cambridge Computer Laboratory Systems Research Group (2013) as post-doctor researchers. He is currently a full professor with the School of Computer Science and Technology, Tongji University, China (since 2014). His research interests include mobile computing and spatiotemporal data science, and he is a member of the CCF and ACM.

Yu Xiao (Member, IEEE) received the doctoral degree in computer science from Aalto University, in 2012. She is currently an associate professor with the Department of Communications and Networking, Aalto University. Her current research interests include edge computing, wearable sensing, and extended reality.

Keshuang Tang received the PhD degree in transportation engineering from Nagoya University, in 2008. He was a post-doctoral research fellow with the University of Tokyo. He was a project assistant professor with Tohoku University. He is currently a professor with the Department of Transportation Information and Control Engineering, Tongji University, China. His main research interests include driver behavior, signal control, and intelligent transportation systems.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on April 23,2026 at 11:13:08 UTC from IEEE Xplore. Restrictions apply.